

Implementacija metoda tabloa za iskaznu logiku

Jovan Škorić
1030/2024

20. jul 2026.

Sažetak

Ovaj rad opisuje implementaciju metoda analitičkih tabloa za iskaznu logiku. Metod tabloa je postupak odlučivanja koji zadovoljivost i tautološkičnost formule utvrđuje sistematskim pokušajem konstrukcije (kontra)modela, razlaganjem formule na potformule po jednoobraznim α/β pravilima. Predstavljamo teorijsku osnovu metoda, opisujemo implementaciju u programskom jeziku C++ i ilustrujemo rad programa na karakterističnim primerima.

1 Uvod

Automatsko rezonovanje bavi se algoritmima koji mehanički utvrđuju da li iz datih pretpostavki logički sledi neki zaključak. U srži mnogih takvih algoritama nalazi se ispitivanje *zadovoljivosti* i *valjanosti* formula. Za iskaznu logiku ovi problemi su odlučivi, ali naivno ispitivanje svih valuacija zahteva 2^n koraka za formulu sa n iskaznih slova, pa se troši rad i na grane koje su očigledno beznačajne.

Metod analitičkih tabloa (engl. *analytic tableaux*) valjanost formule ispituje sistematskim pokušajem konstrukcije *kontramodela* — valuacije koja formulu obara. Umesto nabiranja svih valuacija, formula se razlaže na potformule po malom skupu jednoobraznih pravila, čime se beznačajne grane pretrage rano odsecaju. Ako nijedan kontramodel ne postoji, formula je valjana. Metod je u današnjem obliku uveo Smullyan [1], oslanjajući se na ranije radove Beta i Hintike, a danas je jedna od standardnih tehnika u automatskom dokazivanju teorema [2, 3]. Prednost u odnosu na metod rezolucije jeste u tome što tablo radi neposredno nad izvornom formulom, bez prethodnog prevođenja u normalnu formu, i što se prirodno uopštava na predikatsku i modalne logike.

Ostatak ovog rada organizovan je na sledeći način. U poglavlju 2 navodimo osnovne pojmove iskazne logike i notaciju. U poglavlju 3 dajemo opis metoda tabloa. Poglavlje 4 sadrži detalje implementacije. Najзад, poglavlje 5 sadrži zaključna razmatranja.

2 Osnove

U ovom poglavlju uvodimo sintaksu i semantiku iskazne logike, kao i pojmove zadovoljivosti, valjanosti i SAT problema, koji se koriste u ostatku rada.

Sintaksa. Neka je $\mathcal{P} = \{p, q, r, \dots\}$ prebrojiv skup *iskaznih slova* (atoma). Skup *iskaznih formula* definiše se induktivno:

- logičke konstante $\top$ (tačno) i $\perp$ (netačno) su formule;
- svako iskazno slovo $p \in \mathcal{P}$ je formula;
- ako su A i B formule, onda su i $\neg A$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$ i $(A \leftrightarrow B)$ formule.

Potformula formule je svaka formula upotrebljena u njenoj izgradnji.

Semantika. *Valuacija* je preslikavanje $v : \mathcal{P} \rightarrow \{0, 1\}$ koje svakom iskaznom slovu dodeljuje istinitosnu vrednost. Ono se jednoznačno proširuje do preslikavanja I_v definisanog nad svim formulama:

$$\begin{aligned} I_v(\top) &= 1, & I_v(\perp) &= 0, \\ I_v(p) &= v(p), & I_v(\neg A) &= 1 - I_v(A), \\ I_v(A \wedge B) &= \min(I_v(A), I_v(B)), & I_v(A \vee B) &= \max(I_v(A), I_v(B)), \\ I_v(A \rightarrow B) &= \max(1 - I_v(A), I_v(B)), & I_v(A \leftrightarrow B) &= 1 \text{ akko } I_v(A) = I_v(B). \end{aligned}$$

Ako je $I_v(A) = 1$, kažemo da je formula A *tačna* pri valuaciji v i da je v *model* formule A , u oznaci $v \models A$.

Zadovoljivost i valjanost. Formula je *zadovoljiva* ako ima bar jedan model; u suprotnom je *nezadovoljiva* (kontradikcija). Formula A je *valjana* (tautologija) ako je tačna pri svakoj valuaciji, u oznaci $\models A$. Ova dva pojma povezuje dualnost koju metod tabloa neposredno koristi:

$$\models A \quad \text{akko} \quad \neg A \text{ je nezadovoljiva.} \quad (1)$$

SAT problem. *Problem iskazne zadovoljivosti* (SAT) jeste problem odlučivanja da li je data iskazna formula zadovoljiva. To je prvi problem za koji je dokazano da je NP-kompletna (Cook–Levin), pa se za njega ne očekuje algoritam polinomijalne složenosti. Ispitivanje valjanosti je, po dualnosti (1), komplementaran problem i svodi se na SAT preko negacije formule.

3 Opis metode

Metod tabloa valjanost formule svodi na pokušaj konstrukcije kontramodela. Da bi se u toku konstrukcije pratile pretpostavke o istinitosnim vrednostima potformula, formule se *označavaju*.

Označene formule. *Označena formula* je izraz oblika $T\varphi$ ili $F\varphi$, gde je φ iskazna formula. Oznaka T predstavlja pretpostavku da je φ tačna, a F da je netačna; formalno, uz valuaciju v važi $I_v(T\varphi) = I_v(\varphi)$ i $I_v(F\varphi) = 1 - I_v(\varphi)$. Dokaz da je formula A tautologija počinje od korena FA : pretpostavimo da A nije tačna i pokušamo tu pretpostavku da razvijemo do konkretne valuacije.

Pravila razlaganja. Sva pravila svode se na dva tipa. *Konjunktivna* (α) pravila proširuju granu sa *obe* komponente, dok *disjunktivna* (β) pravila granu *račvaju* na dve. Ova jednoobrazna (Smuljanova) notacija data je u sledećim tabelama:

α	α_1	α_2	β	β_1	β_2
$T(A \wedge B)$	TA	TB	$F(A \wedge B)$	FA	FB
$F(A \vee B)$	FA	FB	$T(A \vee B)$	TA	TB
$F(A \rightarrow B)$	TA	FB	$T(A \rightarrow B)$	FA	TB
$T\neg A$	FA				
$F\neg A$	TA				

Ekvivalenciju razlažemo takođe kao β pravilo, ali sa parovima komponenti: $T(A \leftrightarrow B)$ grana se na $\{TA, TB\}$ i $\{FA, FB\}$, a $F(A \leftrightarrow B)$ na $\{TA, FB\}$ i $\{FA, TB\}$. Za logičke konstante: grana koja sadrži $F\top$ ili $T\perp$ odmah se zatvara, dok se $T\top$ i $F\perp$ uvek zadovoljavaju i uklanjaju.

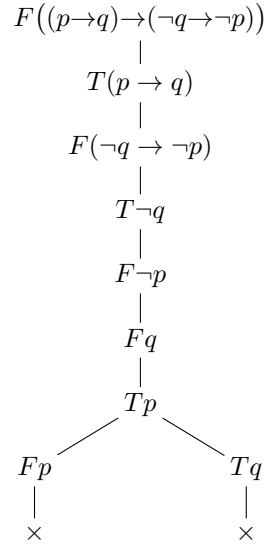
Konstrukcija i zatvaranje. Tablo je stablo označenih formula. Gradi se tako što se na tekućoj grani bira još neiskorišćena formula i na nju primeni odgovarajuće α ili β pravilo, pri čemu se nove formule dopisuju na kraj svih grana koje kroz nju prolaze. Grana se *zatvara* čim sadrži $T\varphi$ i $F\varphi$ za istu formulu φ (očigledna protivrečnost), a *otvorena* je ako je kompletirana bez takve protivrečnosti. Tablo je *zatvoreno* ako su mu sve grane zatvorene. Odatle slede dve procedure odlučivanja:

- **valjanost** formule A : koren je FA ; ako se tablo zatvori, A je tautologija, a inače svaka otvorena grana daje kontramodel;
- **zadovoljivost** formule A : koren je TA ; svaka otvorena grana daje model, a ako se tablo zatvori, A je nezadovoljiva.

Radi efikasnosti, α pravila primenjuju se pre β pravila: pošto α ne račva stablo, odlaganje račvanja daje manji tablo.

Primer. Slika 1 prikazuje zatvoreno tablo za formulu $(p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$ (zakon kontrapozicije). Koren je označen sa F ; uzastopnom primenom α pravila dobijaju se literali Fq i Tp , a razlaganje $T(p \rightarrow q)$ (jedino β pravilo) račva stablo na Fp i Tq . Obe grane sadrže protivrečan par (Tp/Fp , odnosno Fq/Tq), pa se zatvaraju. Kako se ceo tablo zatvorio, formula je tautologija.

Zaustavljanje, korektnost i kompletnost. Svako pravilo zamenjuje označenu formulu njenim pravim potformulama, pa broj veznika strogo opada; zato je svaka konstrukcija konačna. *Korektnost*: ako se tablo sa korenom FA zatvori, formula $\neg A$ je nezadovoljiva, pa je A valjana. *Kompletnost*: svaka otvorena završena grana čini *Hintikin skup*, koji je uvek zadovoljiv; stoga, ako je A tautologija, nijedna grana ne može ostati otvorena, tj. tablo se nužno zatvara. Detaljni dokazi ovih tvrđenja dati su u [1, 2].



Slika 1: Zatvoren tablo za zakon kontrapozicije. Simbol $\times$ označava zatvorenu granu.

4 Implementacija

Jezik i alati. Metod je implementiran u programskom jeziku C++ (standard C++17), bez ikakvih spoljašnjih biblioteka — koristi se isključivo standardna biblioteka. Za prevođenje su dovoljni prevodilac `g++` i `GNU make`.

Organizacija koda. Kôd je podeljen na module prema odgovornosti:

- `formula.hpp` — apstraktno sintaksno stablo (AST) formule;
- `parser.{hpp,cpp}` — leksička i sintaksna analiza ulaza;
- `signed_formula.hpp` — označene formule i α/β klasifikacija;
- `tableau.{hpp,cpp}` — konstrukcija tabloa i procedure odlučivanja;
- `main.cpp` — komandnolinijski interfejs.

Reprezentacija formule. Formula je predstavljena kao `std::variant` pet slučajeva (`False`, `True`, `Atom`, `Not`, `Binary`), a čvorovi se dele preko pametnog pokazivača `std::shared_ptr` (tip `FormulaPtr`), čime se izbegava ručno upravljanje memorijom. Pomoćne funkcije `is<T>` i `as<T>` obavljaju bezbedno ispitivanje i izdvajanje varijante.

Parser. Ulaz se najpre deli na tokene (`tokenize`), a zatim se rekurzivnim spustom gradi AST. Prioritet veznika kodiran je redosledom poziva funkcija: $\neg$ (najviši), pa $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$ (najniži); pri tom su $\rightarrow$ i $\leftrightarrow$ desno, a $\wedge$ i $\vee$ levo asocijativni. Neispravan ulaz signalizira se izuzetkom `ParseError`, koji se hvata na granici sistema, u `main.cpp`.

Klasifikacija pravila. Funkcija `classify` preslikava označenu formulu u strukturu `Expansion`, čije polje `kind` uzima jednu od vrednosti `Alpha`, `Beta`, `Literal`, `Closed` ili `Satisfied`. Time je cela tabela pravila iz poglavlja 3 sabijena u jednu funkciju, pa ostatak algoritma ne mora da pozna je pojedinačne veznike.

Jezgro algoritma. Zatvaranje jedne grane ispituje rekurzivna funkcija `closes`. Ona održava radnu listu složenih formula (`work`) i delimičnu valuaciju literala (tip `Valuation`, tj. `std::map<std::string, bool>`). Pomoćna funkcija `place` smešta formulu na granu: literal upisuje u valuaciju i pri tom otkriva protivrečnost (isti atom sa suprotnim znakom), dok složene formule odlaže u `work`. Funkcija `pickIndex` bira sledeću formulu tako da α pravila imaju prednost nad β , čime se stablo drži malim. Kod β račvanja grana se zatvara samo ako se *sve* opcije zatvore; čim se naide na otvorenu granu, njena valuacija vraća se kao (kontra)model. Javne funkcije `isValid` i `isSatisfiable` postavljaju odgovarajući koren (Ff odnosno Tf) i pozivaju jezgro, a `printValidityTableau` i `printSatisfiabilityTableau` ispisuju celo stablo radi ilustracije.

Prevođenje i pokretanje. Projekat se prevodi komandom `make`, a program se poziva ovako:

```
make                                # prevodi u build/tablo
./build/tablo valid "(p -> q) -> (~q -> ~p)"
./build/tablo sat "(p | q) & (~p | r)" --tree
./build/tablo                        # ugradjeni demo primeri
```

Režim `valid` ispituje tautologičnost, `sat` zadovoljivost, a opcija `--tree` dodatno iscrtaava konstruisani tablo. Za prevođenje je potreban prevodilac sa podrškom za C++17 (npr. `g++` verzije 7 ili novije) i GNU `make`.

Primeri korišćenja. Sledeći isečak prikazuje tipičan rad programa:

```
$ ./build/tablo valid "(p -> q) -> (~q -> ~p)"
(p -> q) -> (~q -> ~p) => TAUTOLOGIJA

$ ./build/tablo valid "p -> q"
p -> q => NIJE TAUTOLOGIJA, kontramodel {p=1, q=0}

$ ./build/tablo sat "(p | q) & (~p | r)"
(p | q) & (~p | r) => ZADOVOLJIVA, model {p=1, r=1}

$ ./build/tablo sat "p -> q" --tree
p -> q => ZADOVOLJIVA, model {p=0}

T (p -> q)
|-- F p
|   o (otvorena) {p=0}
'-- T q
    o (otvorena) {q=1}
```

Prva dva poziva ilustruju dualnost: za formulu koja nije tautologija program vraća kontramodel koji je obara. U poslednjem primeru opcija `--tree` prikazuje

konstruisani tablo — koren $T(p \rightarrow q)$ je β formula, pa se grana na Fp i Tq ; obe grane ostaju otvorene i daju po jedan model, a oznaka o obeležava otvorenu granu.

5 Zaključak

U ovom radu implementiran je metod analitičkih tabloa za iskaznu logiku i opisana je njegova teorijska osnova. Program, napisan u jeziku C++, za proizvoljnu iskaznu formulu odlučuje njenu tautologičnost i zadovoljivost i, po potrebi, konstruiše (kontra)model ili ispisuje ceo tablo.

Glavna prednost metoda jeste to što radi neposredno nad izvornom formulom, bez prevođenja u normalnu formu, i što se otvorena grana prirodno čita kao model. Kao i svi postupci za SAT, metod u najgorem slučaju ima eksponencijalnu složenost; prikazana implementacija koristi osnovnu heuristiku prioriteta α nad β pravilima, dok naprednije varijante (na primer KE-tablo ili sprega sa DPLL pretragom) mogu dodatno smanjiti veličinu stabla. Budući da se pravila razlaganja lako uopštavaju, isti okvir prirodno se proširuje na predikatsku i modalne logike, što metod tabloa čini korisnim i izvan iskazne logike.

Literatura

- [1] Smullyan, Raymond M. *First-Order Logic*. Springer-Verlag, 1968.
- [2] Fitting, Melvin. *First-Order Logic and Automated Theorem Proving*. 2nd ed., Springer, 1996.
- [3] D’Agostino, Marcello, et al., eds. *Handbook of Tableau Methods*. Kluwer Academic Publishers, 1999.